WHALE PINBALL

I created a game called "Whale Balance Ball", using arduino combined with Unreal Engine, utilizing the capacitive touch pins that come with the ESP32, and communicating with the UE5 through the UDP protocol, I realized a balance ball game where we can control the whale in the game by touching the fruits.

By clicking the fruits we can control the movement of the whale in the game. Since the ESP32 comes with capacitive touch pins, you can freely change the fruits used for control, which can be apples, oranges, mangoes or even small tomatoes.

PROJECT DESCRIPTION

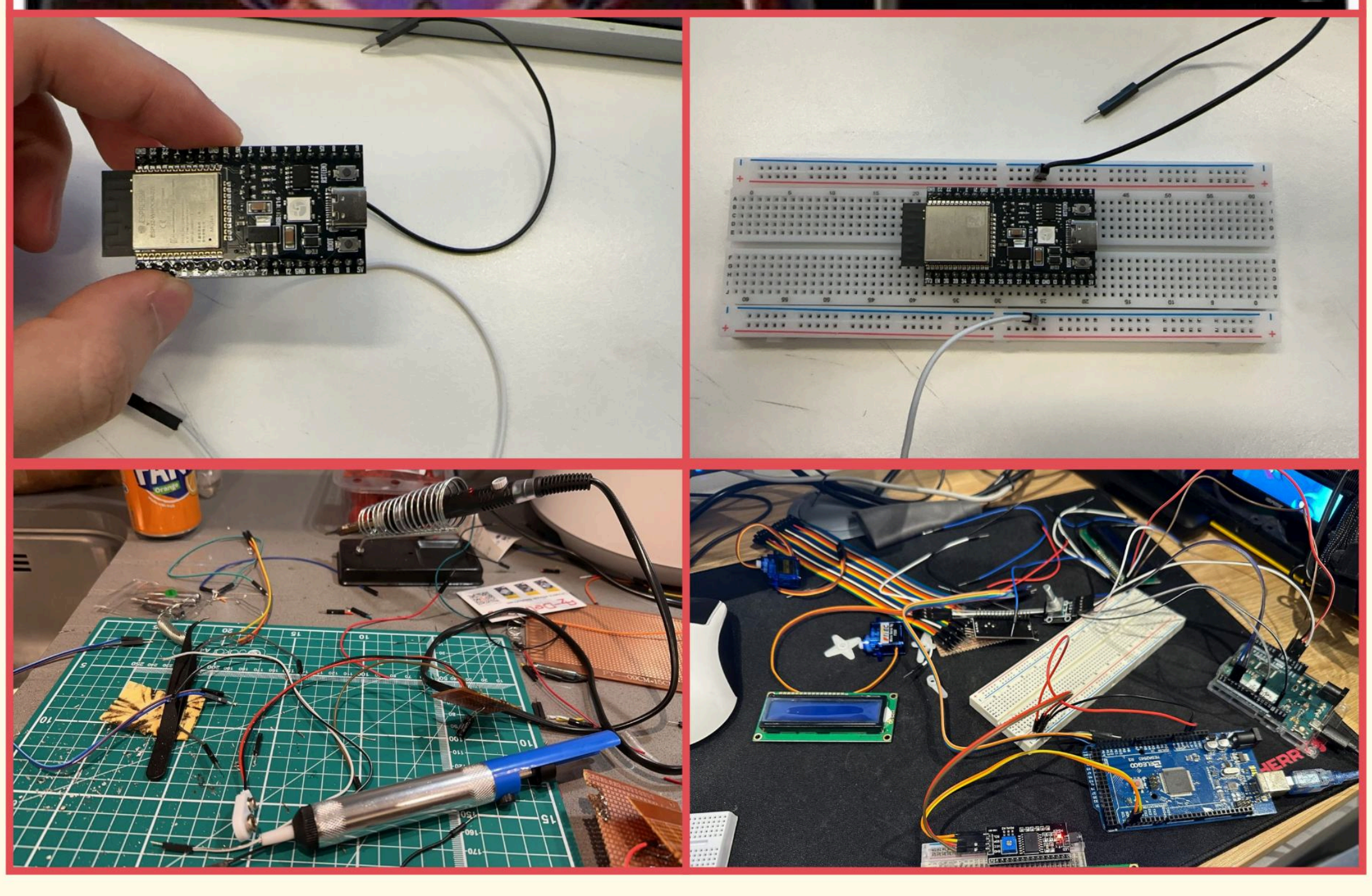
WHALE PINBALL INSPIRATION

The inspiration for this project actually comes from my ongoing exploration of the connection between the physical world and digital space while using the ESP32 and UE platforms. In my learning and practice, I discovered that many Arduino projects use everyday objects as concrete control methods. This skeuomorphic design approach made me realize that simple, intuitive elements—such as apples and oranges—can help users understand complex control logic. At the same time, I was also inspired by the "behavior-triggered feedback" pattern commonly found in interactive installation art, especially in how data is transformed from perceived behavior into visual action. I combined these technical clues with creative methods, resulting in a work that retains both the logical clarity of programming and engineering, and possesses an approachable and engaging visual language.

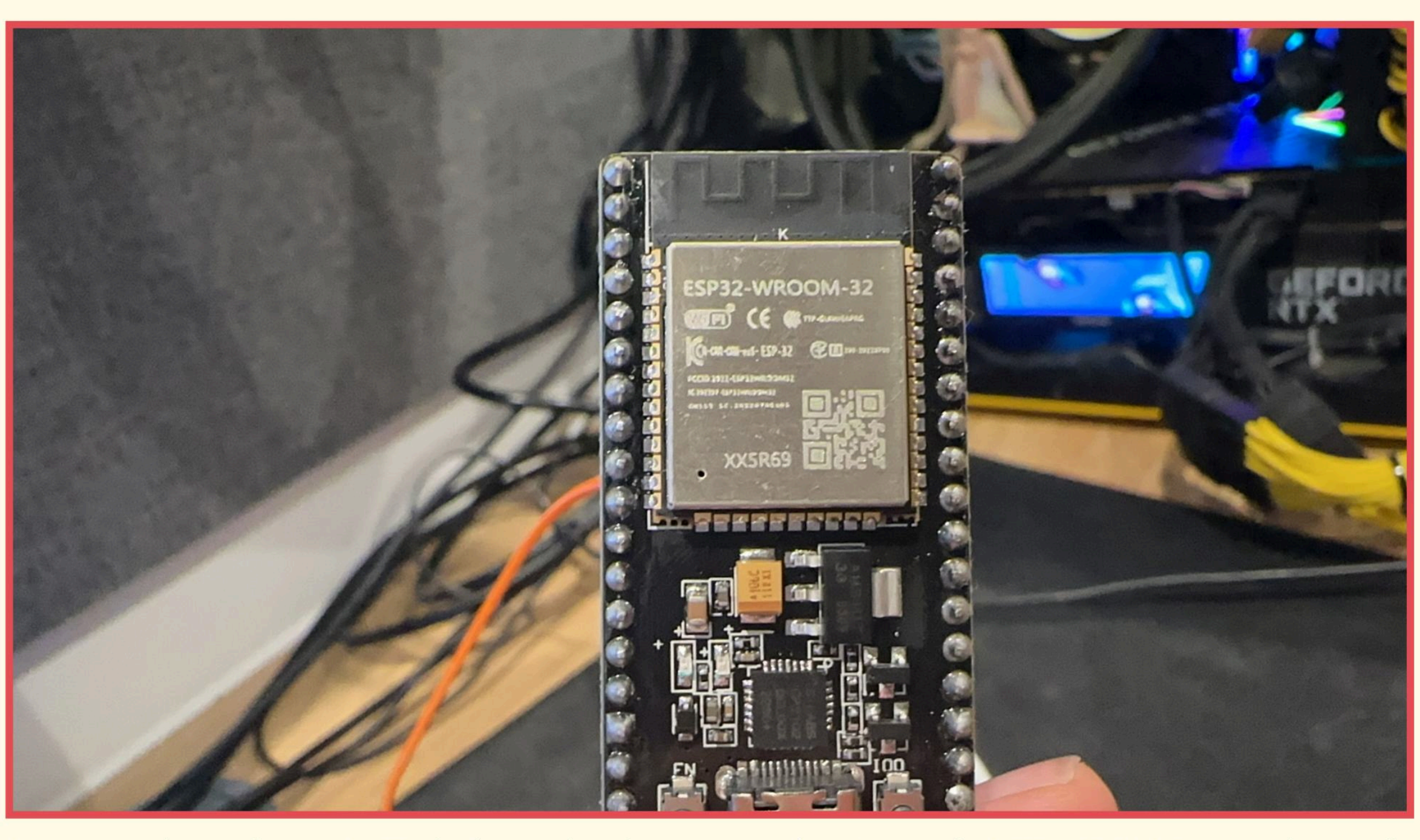
In this project, I aimed to achieve real-time interaction between a physical device and a virtual character through UDP communication between the ESP32 development board and UE5. The core of the design lies in converting the physical input signals read from the capacitive touch pins into visual feedback in the virtual world. To make the control process clearer and more intuitive, I used "apple" and "orange" as representatives of the two characters, constructing corresponding static meshes in UE5 and controlling their displacement along the Z-axis after receiving a signal using blueprints. This mapping method allows me to drive behavior in virtual scenes with simple physical actions, thus establishing an effective cross-media interaction mechanism. The entire process is broken down into three clear modules: input (touch), transmission (UDP), and feedback (character movement), which facilitates maintaining the system's stability and controllability during later debugging and optimization.

In terms of gameplay, I referenced the game mode of 3D pinball. I set up a one-to-one mapping relationship between physical touch and virtual character movements through different configurations of capacitive touch pins. Whenever I touch a designated pin on the development board, the corresponding character in UE5 will make a corresponding movement feedback. This operation method constructs a "pinball"-style interaction mechanism. Although the character currently only makes simple Z-axis displacements in space, this reaction itself has preliminary game-like qualities: every physical action of the user can be instantly converted into visual changes, forming a continuous interactive loop. This mechanism not only allows me to better understand the signal logic of UDP communication but also lays the foundation for possible future gameplay expansions (such as controlling multiple characters or embedding different level logic).

If you remember this the you've been through the "no internet" struggle.
3D Pinball
Space Cadet
BALL 1
1 1703750
Wormhole Opened
Drop Targets Left: 8

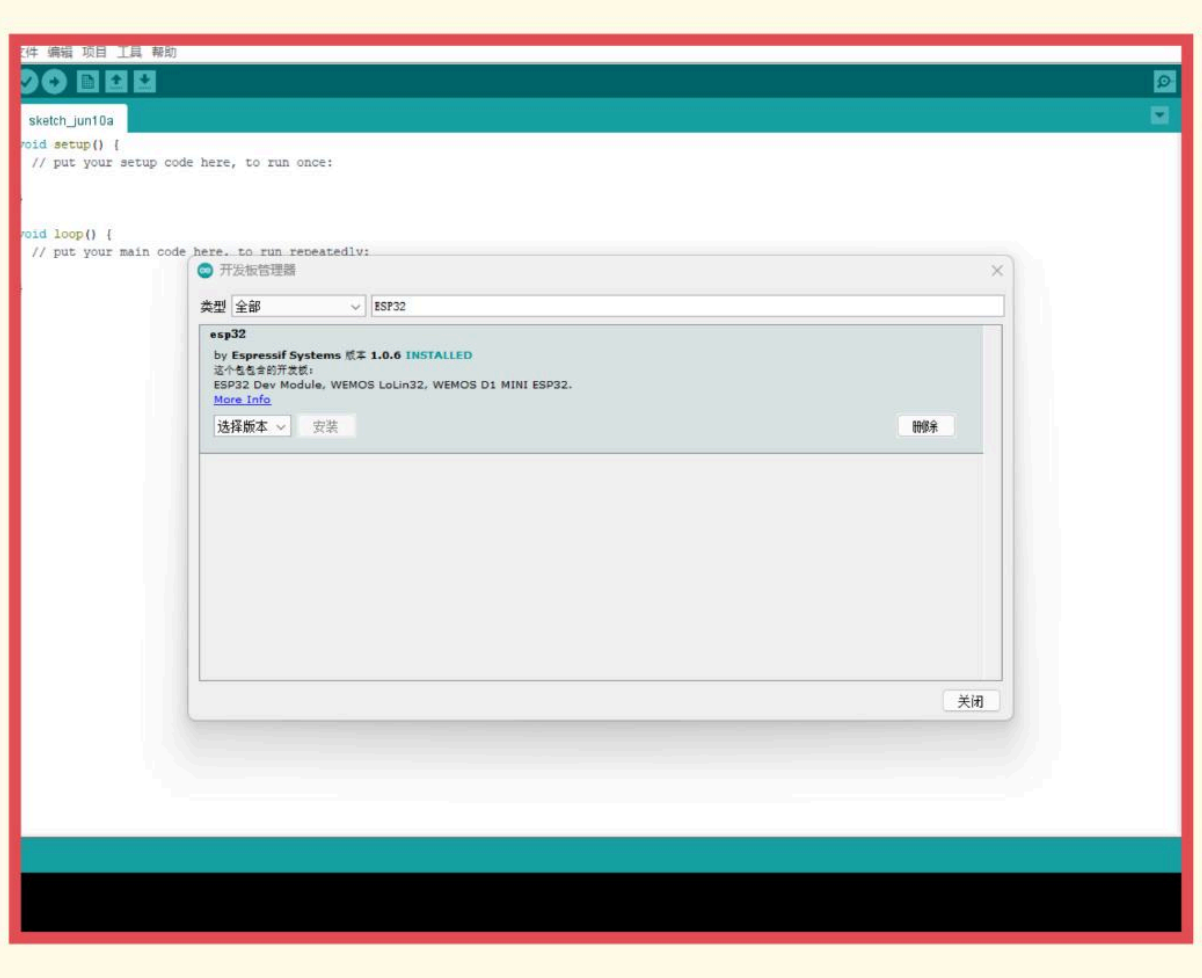


WHALE PINBALL PROCESS



First, you need to purchase an ESP32 development board. You can see that I'm using the ESP32-WROOM-32 version because the ESP32 has a built-in capacitive touch pin, and its Wi-Fi function allows for UDP communication with the UE5, so you can receive signals from the development board through the UE5's internal computer port. Of course, if you're using a UE4, you can also connect it directly to the Arduino via a plugin, such as UE4Duino.

After connecting the ESP32 to your computer, you need to open Arduino, find the Board Manager, search for esp32, and then click "Install". If you can't find esp32 here, you need to enter this URL in the Board Manager URL field in preferences: https://dl.espressif.com/dl/package_esp32_index.json, and then repeat the installation steps above.

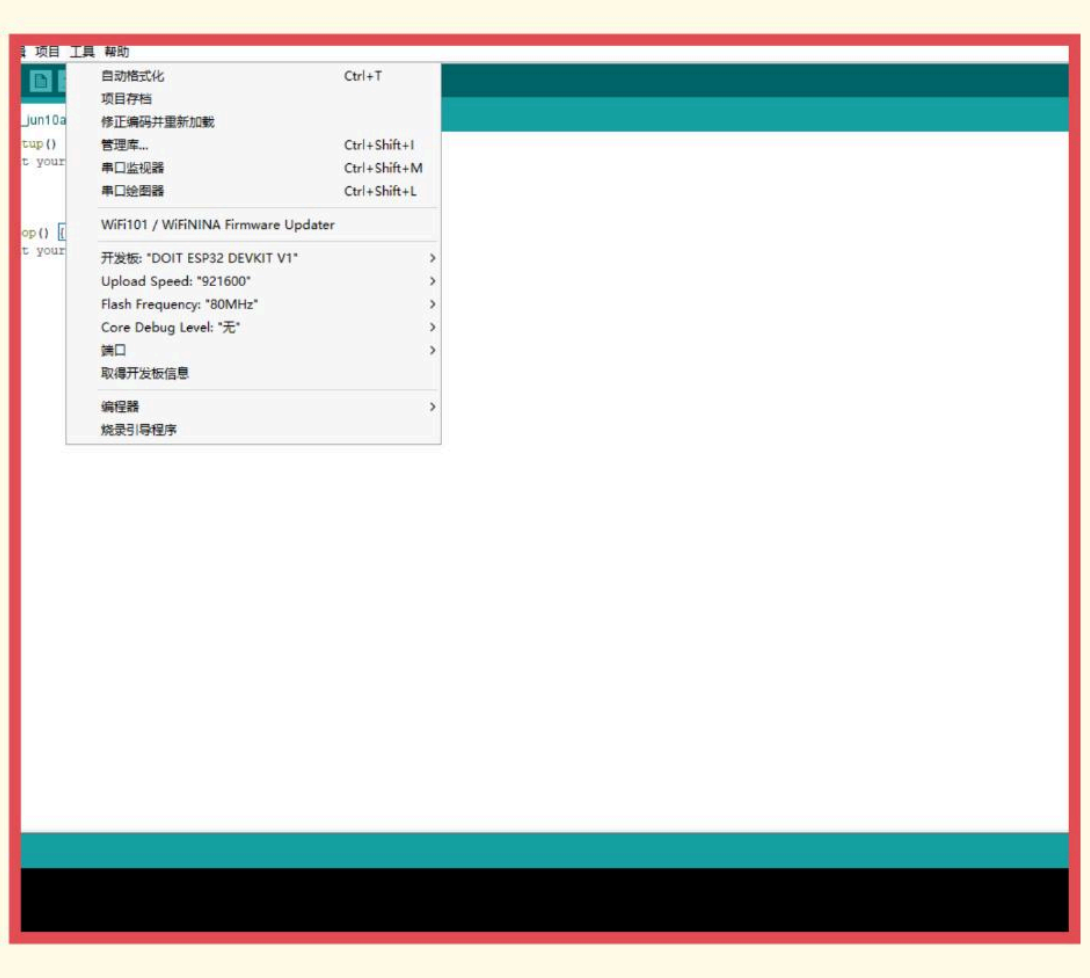


```
void setup() {
  // put your setup code here, to run once:

void loop() {
  // put your main code here, to run repeatedly:
```

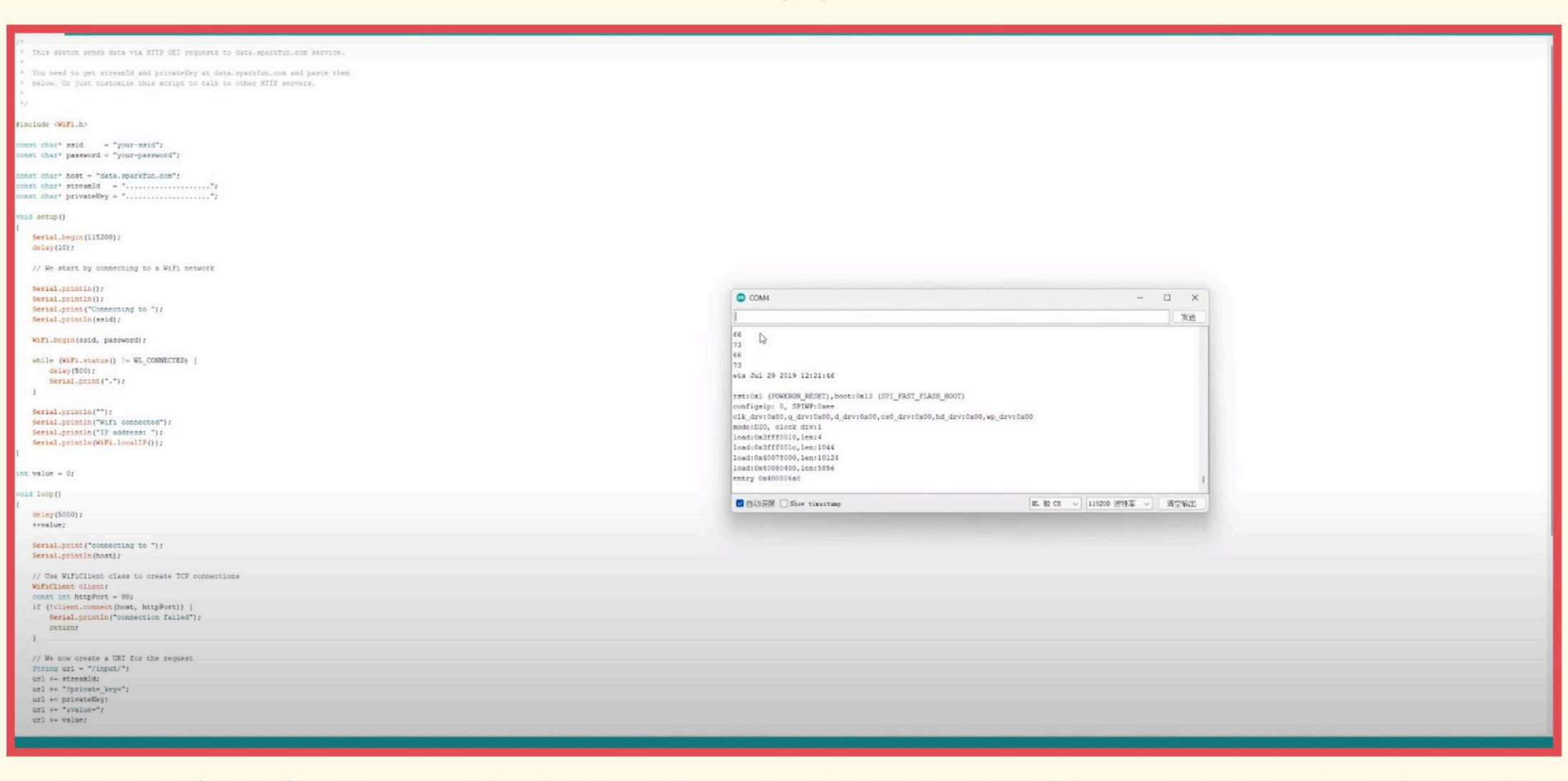
开发板管理器

类型 全部 ESP32

esp32
by Espressif Systems 版本 1.0.6 INSTALLED
这个包包含的开发板:
ESP32 Dev Module, WEMOS LoLin32, WEMOS D1 MINI ESP32.
More Info



Next, you need to test the ESP32's Wi-Fi functionality. Locate the Wi-Fi option in the example, enter it, and select the Wi-Fi client. You'll need to enter your Wi-Fi name and password in these two lines. After entering them, upload the code to the development board. Once uploaded successfully, open the serial monitor and press the reset button on the development board. After a successful connection, the serial monitor will display network information.

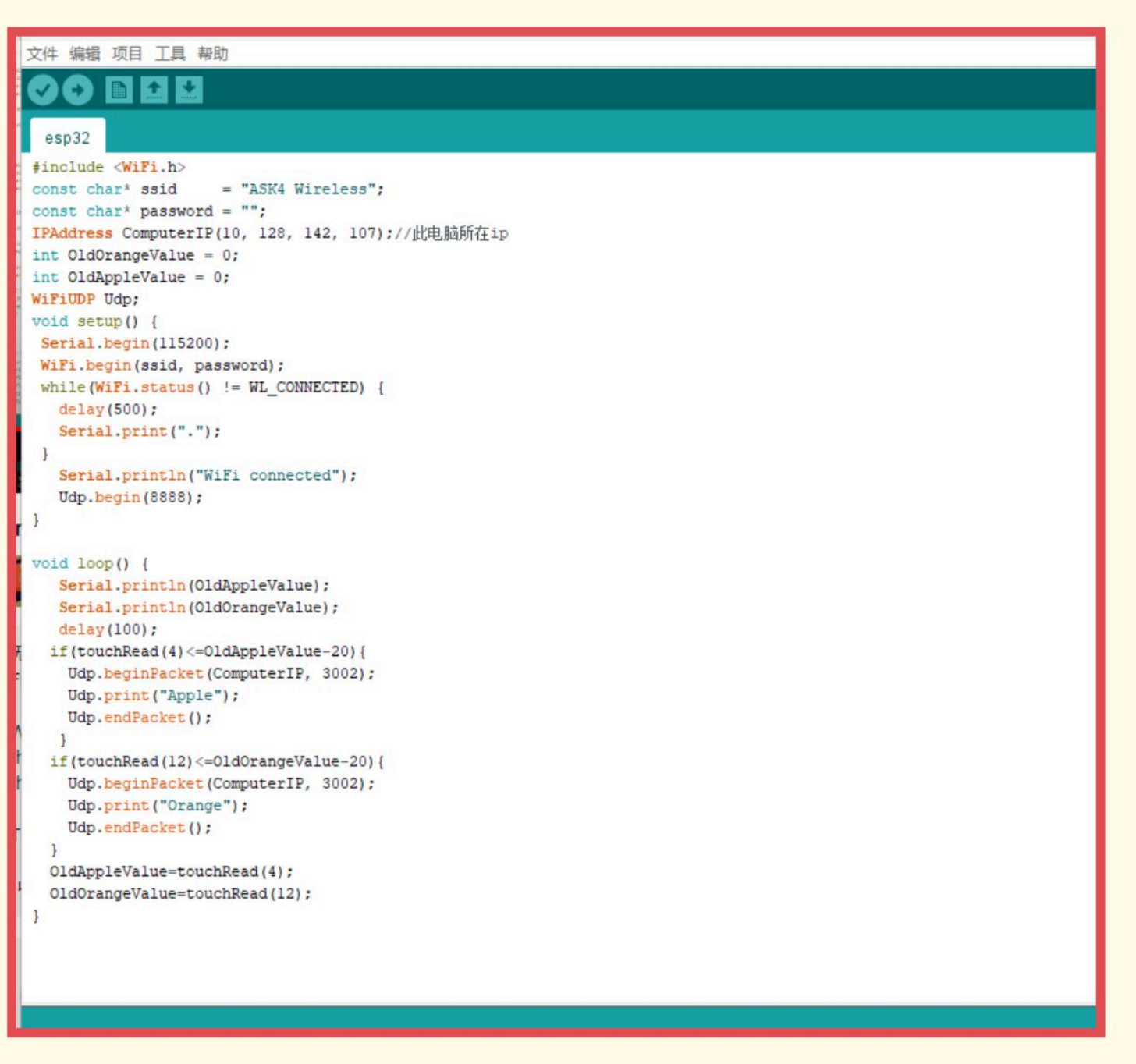


```
/*
 *  This sketch sends data via HTTP GET requests to data.sparkfun.com service.
 *
 *  You need to get streamId and privateKey at data.sparkfun.com and paste them
 *  below. Or just customize this script to talk to other HTTP servers.
 *
 */

#include <WiFi.h>

const char* ssid     = "your-ssid";
const char* password = "your-password";

const char* host = "data.sparkfun.com";
const char* streamId   = "....................";
const char* privateKey = "....................";

void setup()
{
    Serial.begin(115200);
    delay(10);

    // We start by connecting to a WiFi network

    Serial.println();
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
}

int value = 0;

void loop()
{
    delay(5000);
    ++value;

    Serial.print("connecting to ");
    Serial.println(host);

    // Use WiFiClient class to create TCP connections
    WiFiClient client;
    const int httpPort = 80;
    if (!client.connect(host, httpPort)) {
        Serial.println("connection failed");
        return;
    }

    // We now create a URI for the request
    String url = "/input/";
    url += streamId;
    url += "?private_key=";
    url += privateKey;
    url += "&value=";
    url += value;
```

```
66
73
66
73
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0018,len:4
load:0x3fff001c,len:1044
load:0x40078000,len:10124
load:0x40080400,len:5856
entry 0x400806a8
```

Important Note: Do not use Wi-Fi networks that require a second login, such as the public Wi-Fi in your apartment, as this will prevent your motherboard from connecting to the network and thus rendering it unusable, as the motherboard cannot log in to these Wi-Fi networks.

Once everything is ready, we need to modify the code to send UDP signals to the computer via Wi-Fi, and we need to determine the ports on the computer that receive these signals. These ports need to be the same as those used later in UE5. Here, I'm using apples and oranges as controls, so I've entered apples and oranges as values here; you can change them as you like. The computer's IP address can be viewed in the command prompt using the `wifi-config` command. Here, I've set pins 4 and 12 to receive signals from apples and oranges, respectively.



```
#include <WiFi.h>
const char* ssid     = "ASK4 Wireless";
const char* password = "";
IPAddress ComputerIP(10, 128, 142, 107);//此电脑所在ip
int OldOrangeValue = 0;
int OldAppleValue = 0;
WiFiUDP Udp;
void setup() {
 Serial.begin(115200);
 WiFi.begin(ssid, password);
 while(WiFi.status() != WL_CONNECTED) {
   delay(500);
   Serial.print(".");
 }
   Serial.println("WiFi connected");
   Udp.begin(8888);
}

void loop() {
   Serial.println(OldAppleValue);
   Serial.println(OldOrangeValue);
   delay(100);
  if(touchRead(4)<=OldAppleValue-20){
    Udp.beginPacket(ComputerIP, 3002);
    Udp.print("Apple");
    Udp.endPacket();
   }
  if(touchRead(12)<=OldOrangeValue-20){
    Udp.beginPacket(ComputerIP, 3002);
    Udp.print("Orange");
    Udp.endPacket();
  }
  OldAppleValue=touchRead(4);
  OldOrangeValue=touchRead(12);
}
```

CODE FOR ESP32

```
#include <WiFi.h>
const char* ssid     = "Rosso-staff";//Ur wifi
name
const char* password = "rosso888";//Ur wifi
password
IPAddress ComputerIP(10, 128, 142, 107);//
COMPUTER IP
int OldOrangeValue = 0;
int OldAppleValue = 0;
WiFiUDP Udp;
void setup() {
 Serial.begin(115200);
 WiFi.begin(ssid, password);
 while(WiFi.status() != WL_CONNECTED) {
   delay(500);
   Serial.print(".");
 }
   Serial.println("WiFi connected");
   Udp.begin(8888);
}

void loop() {
   Serial.println(OldAppleValue);
   Serial.println(OldOrangeValue);
   delay(100);
  if(touchRead(4)<=OldAppleValue-20){
    Udp.beginPacket(ComputerIP, 3002);//Should
be the same port in ue5
    Udp.print("Apple");
    Udp.endPacket();
   }
  if(touchRead(12)<=OldOrangeValue-20){
    Udp.beginPacket(ComputerIP, 3002);//Should
be the same port in ue5
    Udp.print("Orange");
    Udp.endPacket();
  }
  OldAppleValue=touchRead(4);
  OldOrangeValue=touchRead(12);
}
```

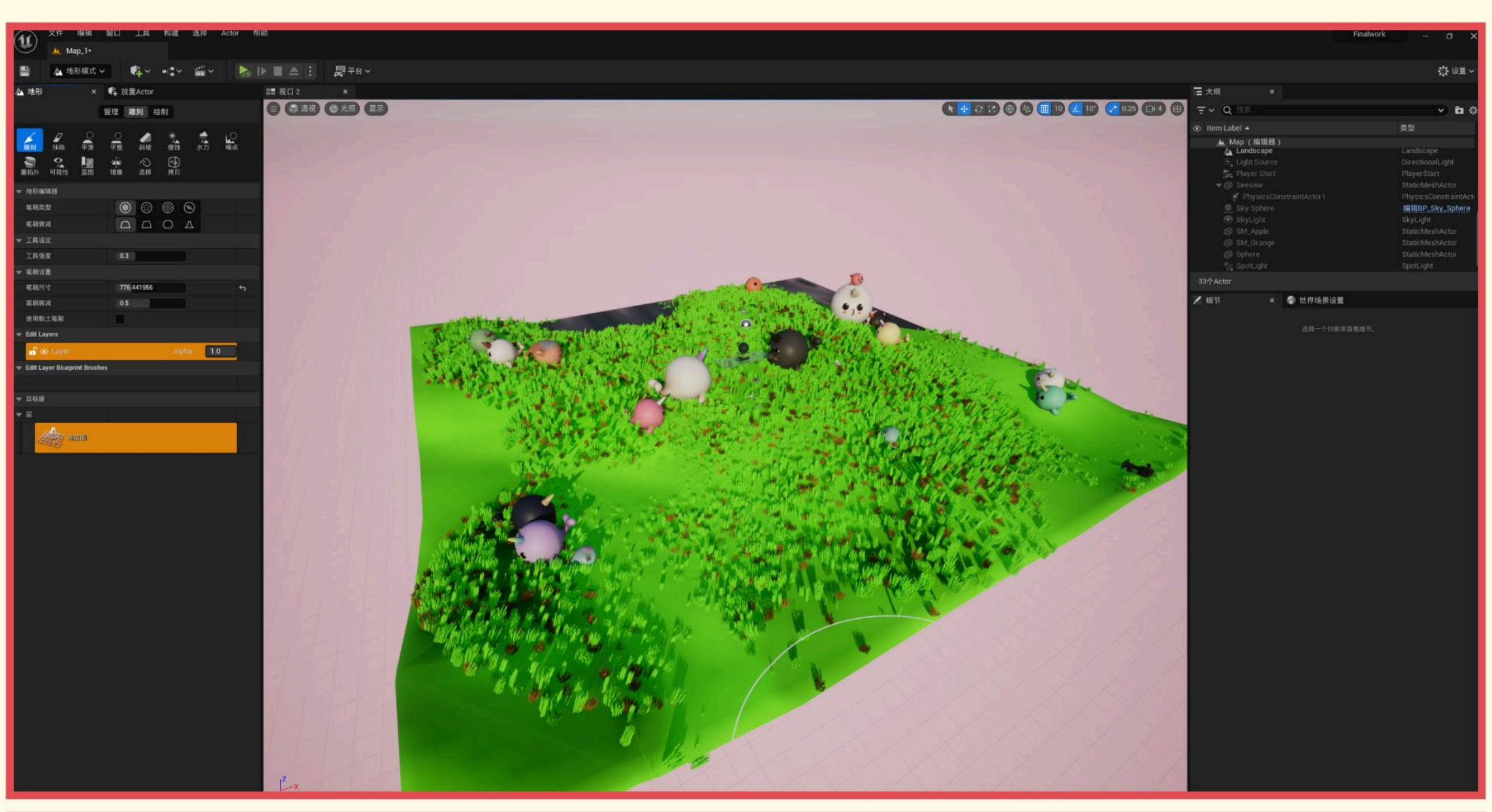
WHALE PINBALL PROCESS

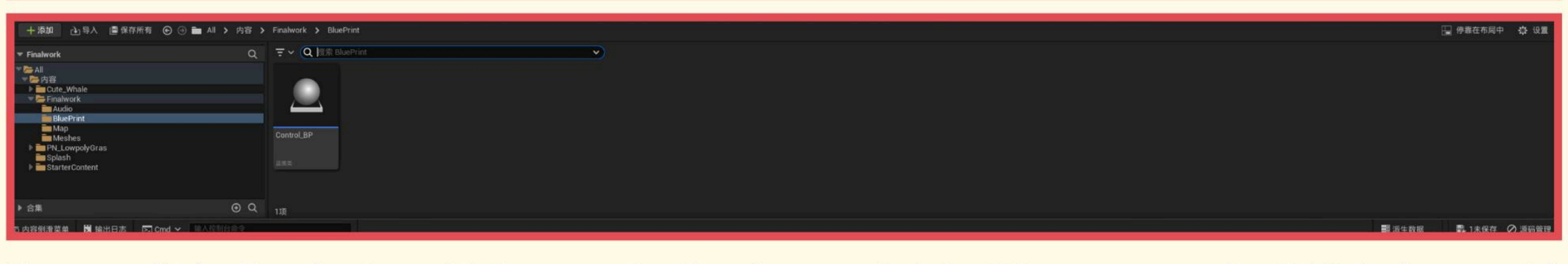
The most crucial step now is to download and install the UDP protocol plugin for UE5. You can find it by searching for UDP-Unreal on GitHub, which is fantastic!

DOWNLOAD URL: getnamo/UDP-Unreal

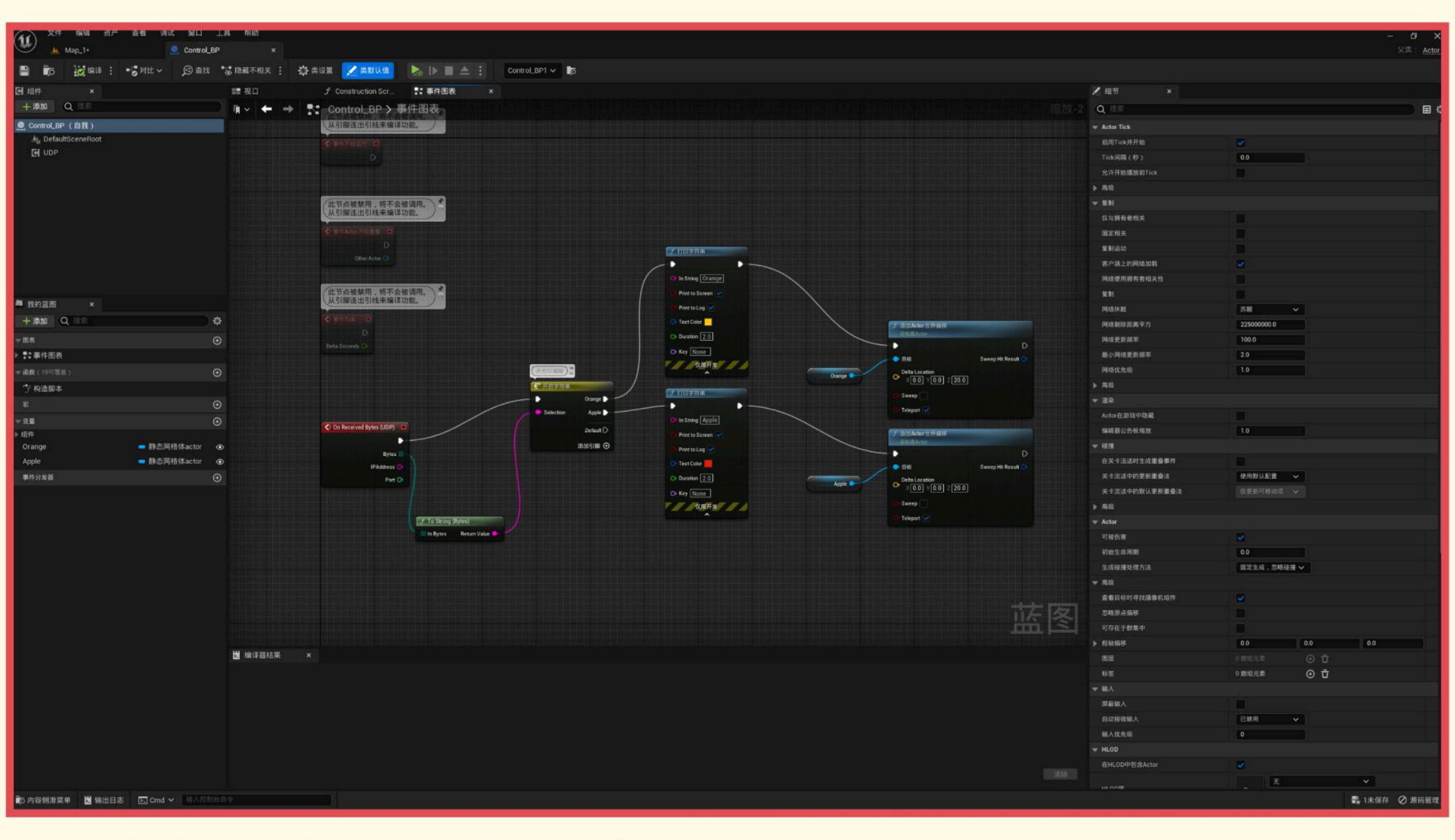
PART 2 UNREAL ENGINE 5 BINDING

Next, in UE5, create your game world. Without going into detail, you'll need to build the scene, characters, and blueprints. Note that I recommend converting the two characters you'll use for control to static meshes to avoid unnecessary problems. Once everything is ready, we need to enable the main character to be controlled via signals. It's important to note that we cannot add components that read UDP signals within the level blueprint, so we need to create a new blueprint, the Control Blueprint (Control-BP).





Next, we edit the blueprint. Press Ctrl+E to enter the blueprint page, find the UDP component under "Add" in the upper left corner, and add it to the blueprint. We also add two static mesh components; here, I've named them Apple and Orange to maintain consistency. Note that we need to keep the same UDP signal port in both Arduino and UE5; here, I've set it to port 3002. Next are some general settings to allow UE5 to provide feedback based on the pins and print different objects when a signal is received. In this example, I've set the bound object to move 20 units along the Z-axis after receiving a signal.



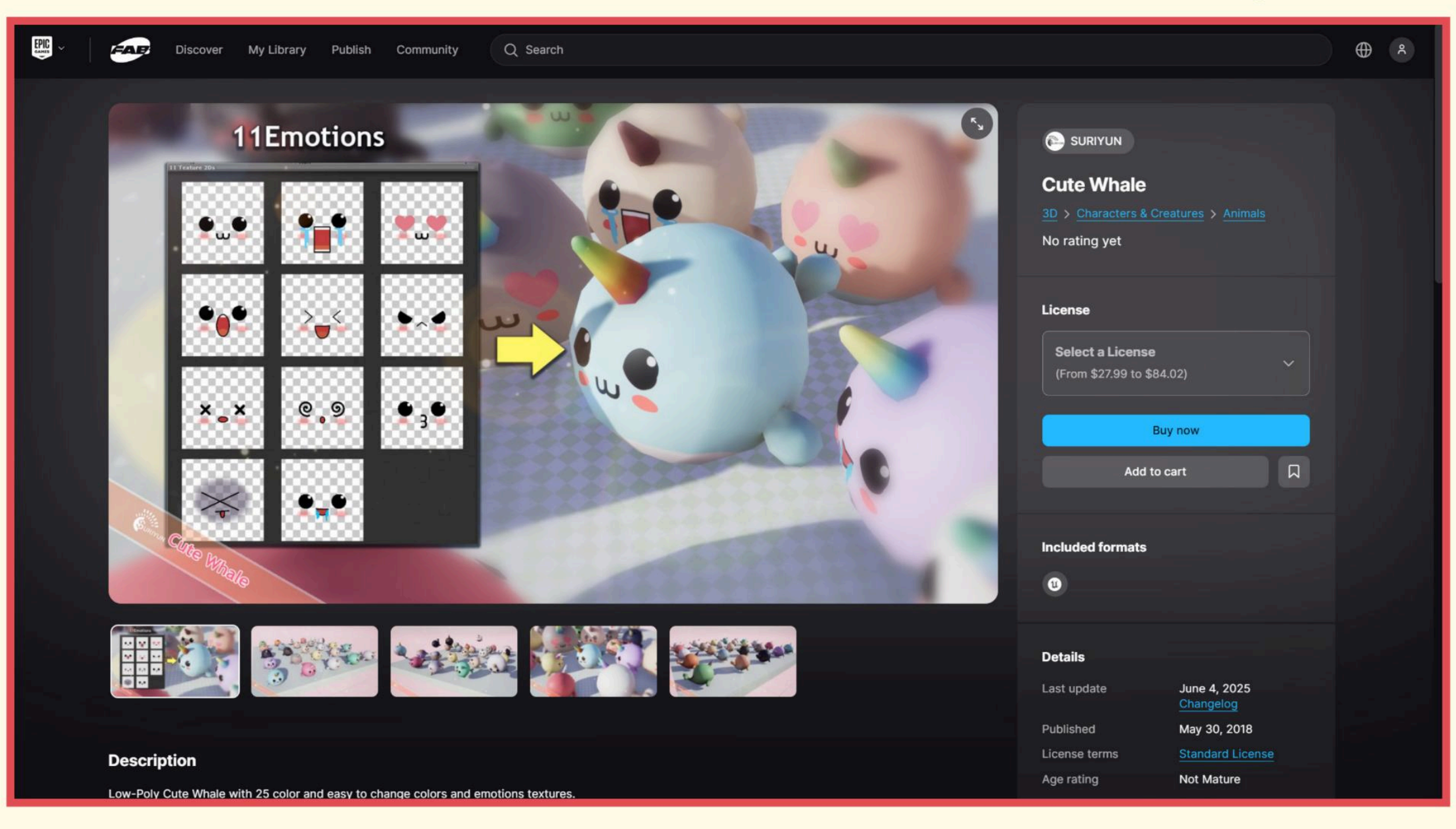
Next, simply place your blueprint within the map and bind it to the two characters you want to control in the bottom right corner details.

That's basically it. Now, you just need to do an overall test to see if the ESP32 can control your characters. Of course, you can also control their movement more freely or add more pins to output signals.

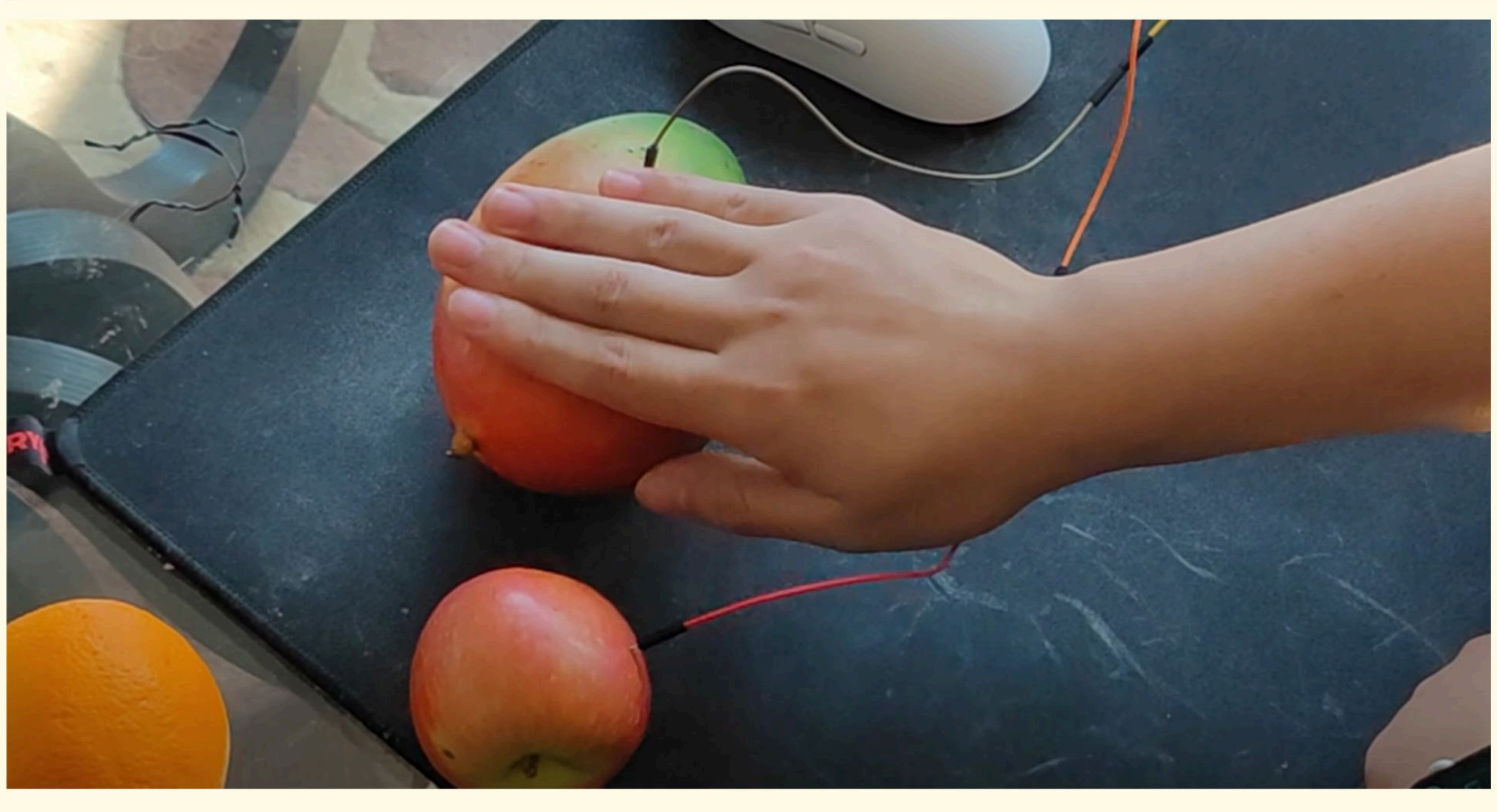
USE OF ASSETS

https://www.fab.com/listings/27c09605-160d-406e-901c-5a1b382d8a09

For the model, I used the CUTE WHALE asset from the Unreal Engine Store.



WHALE PINBALL FINAL DISPLAY



By repeatedly tapping the fruit, you control the character bound to the Unreal Engine to move upwards and maintain balance on the ball in the center.